

Implementation of AFEM

Adaptive Finite Element Methods for Elliptic PDEs — Lecture 6

Pedro Morin Ricardo H. Nochetto



github.com/morinpedro/afem-cimpa-Rosario-2026

From black boxes to code

- **The AFEM loop:** Lecture 5 closed with the formal adaptive loop of four modules,

$SOLVE \rightarrow ESTIMATE \rightarrow MARK \rightarrow REFINE.$

- **This lecture:** realizes each module as a Matlab/Octave function, and the loop is `afem.m`.

What carries over from Lecture 3

SOLVE is exactly the fixed-mesh assembly plus the solving of the resulting linear system

— unchanged, just run on the current mesh.

What is genuinely new

- A **mesh data structure** that answers “who is my neighbour?” in $O(1)$ per element.
- MARK and REFINE — no counterpart on a fixed mesh.

Roadmap

- 1 The adaptive loop, in code
- 2 Mesh data structure for adaptivity
- 3 SOLVE and ESTIMATE
- 4 MARK
- 5 REFINE
- 6 Validation: adaptive vs. uniform
- 7 Source code

The adaptive loop, in code

The driver afem.m

Reads problem data, optionally does a few uniform startup refinements, then iterates the four modules until a stopping criterion fires:

```
[prob_data, adapt] = init_data();
% ... read mesh.elem_vertices, elem_neighbours, elem_boundaries, vertex_coordinates ...
iter_counter = 1;
while (1)
    [uh,fh] = assemble_and_solve(prob_data,mesh,uh,fh);    % (assemble and) SOLVE (Lecture 3, current mesh)
    [est, mesh] = estimate(prob_data, adapt, mesh, uh);    % ESTIMATE -> mesh.estimator, global est
    N_vert = mesh.n_vertices;
    if ((iter_counter >= adapt.max_iterations) || (est < adapt.tolerance) ...
        || (N_vert > adapt.max_vertices))
        break;
    else
        mesh = mark_elements(adapt, mesh);                % MARK -> mesh.mark
        [mesh,uh,fh] = refine_mesh(mesh,uh,fh);            % REFINED -> new conforming mesh
    end
    iter_counter = iter_counter + 1;
end
```

Three stopping conditions: tolerance reached, iteration cap, or a vertex-count safety cap.

The modules

- **SOLVE** (`assemble_and_solve.m`): assembles the linear system and solves it, storing the load vector in `fh` and the solution in `uh`.
- **ESTIMATE** (`estimate.m`): computes, for each element:

$$\mathcal{E}_{\mathcal{T}}(U, T) = \left(C_1 h_T^2 \|r\|_{L^2(T)}^2 + C_2 h_T \|j\|_{L^2(\partial T)}^2 \right)^{1/2},$$

$$\text{with } r = \underbrace{-\operatorname{div}(A \nabla U)}_{=0} + b \cdot \nabla U + cU \quad \text{and} \quad j|_s = \begin{cases} \llbracket A \nabla U \cdot n \rrbracket, & \text{if } s \text{ is interior} \\ A \nabla U \cdot n - g_N, & \text{if } s \text{ is a Neumann side} \end{cases}$$

ESTIMATE populates `mesh.estimator` with one of these per element. Returns `test` with $\mathcal{E}_{\mathcal{T}}(U)$.

- **MARK**: writes an integer refinement count into each element into the vector `mesh.mark`, using a *marking strategy*.
- **REFINE**: refines, using newest vertex bisection, each element at least `mesh.mark(e1)` times, maintaining conformity and rebuilding the connectivity and neighbour information. It also interpolates `uh`, `fh` into the new mesh.

Mesh data structure for adaptivity

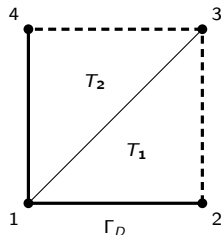
What adaptivity needs beyond Lecture 3

- **Beyond a fixed mesh:** The four text files of Lecture 3 (geometry + boundary vertices) suffice on a *fixed* mesh. Adaptivity needs two more relations, available in $O(1)$ per element.
- **Neighbour across each side:** for the jump term of the estimator, and for propagating refinement.
- **Boundary type of each side:** so Dirichlet/Neumann can be reconstructed on the fly.

Four element-indexed arrays

- `elem_vertices` ($N_t \times 3$),
- `elem_neighbours` ($N_t \times 3$),
- `elem_boundaries` ($N_t \times 3$),
- `vertex_coordinates` ($N_p \times 2$).

The data, on two triangles



elem_vertices				elem_neighbours			
T_1 :	1	2	3	T_1 :	0	2	0
T_2 :	1	3	4	T_2 :	0	1	0
elem_boundaries				vertex_coordinates			
T_1 :	+1	0	+1	1:	0.0	0.0	
T_2 :	-1	0	+1	2:	1.0	0.0	
				3:	1.0	1.0	
				4:	0.0	1.0	

- Side i is opposite vertex i ; side 3 is the **refinement edge**.
- Shared side 2 of both elements $\Rightarrow \text{elem_neighbours}(T_1, 2) = T_2$ and vice versa.
- Boundary flags: +1 Dirichlet, -1 Neumann, 0 interior.

SOLVE and ESTIMATE

SOLVE: nothing new

- **Reused from Lecture 3:** SOLVE is the fixed-mesh assembly of Lecture 3 plus the solving of the resulting linear system (`assemble_and_solve`), applied to whatever the current mesh happens to be.
- **The ingredients:** element stiffness/mass via the reference map, midpoint quadrature for the load, Simpson's rule on Neumann segments, Dirichlet rows replaced by the identity.

Only the mesh changes

Same code, same formulas — the adaptive loop just calls it once per iteration, on a mesh that keeps growing.

ESTIMATE: the residual indicator

- **Residual indicator:** Evaluates $\mathcal{E}_T^2 = C_1 h_T^2 \|r_T\|_{L^2(T)}^2 + C_2 h_T \|J_T\|_{L^2(\partial T)}^2$ of Lecture 5. For $\mathcal{P}_1$ and piecewise-constant coefficients, $D^2 U = 0$ on each element, so

$$r_T = -\operatorname{div}(A \nabla U) + b \cdot \nabla U + c U - f = b \cdot \nabla U + c U - f,$$

and the jump term collects $a \nabla U \cdot \mathbf{n}$ across the three sides.

- **Coefficients per element:** As in `fem.m`, a , b , c are function handles evaluated once per element, at the barycenter.
- **First loop:** constant $\nabla U|_T$, then $a|_T$, $b|_T$, $c|_T$, for every element.
- **Second loop:** interior residual at the three edge midpoints, plus a loop over the three sides accumulating the jump term.

ESTIMATE: the code

The two loops in full:

```

for el = 1:n_elem                                % constant grad U, and a|_T, c|_T, per element
    v = mesh.elem_vertices(el,:); B = [ V(v(2))-V(v(1)) , V(v(3))-V(v(1)) ];
    bary = (V(v(1))+V(v(2))+V(v(3)))/3;
    grduh(el,:) = ( B' \ (grd_bas_fcts*uh(v)) )';
    coeff_a(el) = prob_data.a(bary);    coeff_b(el,:) = prob_data.b(bary);    coeff_c(el) = prob_data.c(bary);
end
for el = 1:n_elem
    ... el_area = abs(det(B))/2;    h_T = sqrt(el_area);
    r12 = coeff_b(el,:)*grduh(el,:)' + coeff_c(el)*uh12 - f12;    % (and r23, r31)
    jump_res2 = 0;
    for side = 1:3
        normal = [tangential(2); -tangential(1)]/norm(tangential);
        if (mesh.elem_boundaries(el,side) == 0)                % interior side
            neigh = mesh.elem_neighbours(el,side);
            j = (coeff_a(el)*grduh(el,:) - coeff_a(neigh)*grduh(neigh,:)) * normal;
        elseif (mesh.elem_boundaries(el,side) < 0)            % Neumann side
            j = coeff_a(el)*grduh(el,:)*normal - prob_data.gN(midpoint);
        end                                                    % Dirichlet side: no jump
        jump_res2 = jump_res2 + j*j';
    end
    mesh.estimator(el) = sqrt( adapt.C(1)*h_T^2*el_area*(r12^2+r23^2+r31^2)/3 ...
                               + adapt.C(2)*h_T *h_T*jump_res2 );
end

```

MARK

Three marking strategies

The three strategies, side by side:

```
function mesh = mark_elements(adapt, mesh)
mesh.mark = zeros(mesh.n_elem, 1);
switch adapt.strategy
case 'GR'           % global (uniform) refinement: mark everything
    mesh.mark = adapt.n_refine * ones(size(mesh.estimator));
case 'MS'           % maximum strategy: mark elements within a fraction of the largest
    mesh.mark(find(mesh.estimator > adapt.MS_gamma*mesh.max_est)) = adapt.n_refine;
case 'Doerfler'     % Doerfler (bulk) strategy
    threshold = adapt.Doerfler_theta^2 * mesh.est_sum2;
    gamma = 1;  est_sum2_marked = 0;
    while (est_sum2_marked < threshold)
        gamma = gamma - adapt.Doerfler_nu;
        f = find(mesh.estimator > gamma*mesh.max_est);
        mesh.mark(f) = adapt.n_refine;
        est_sum2_marked = sum(mesh.estimator(f).^2);
    end
end
```

mesh.mark entries are the *number of times* each element is bisected (adapt.n_refine, typically 2).

Dörfler marking, realized

The bulk criterion

$$\mathcal{E}(U, \mathcal{M}) \geq \theta \mathcal{E}(U, \mathcal{T}), \quad 0 < \theta < 1, \quad \mathcal{E}(U, \mathcal{M}) = \left(\sum_{T \in \mathcal{M}} \mathcal{E}_T^2 \right)^{1/2},$$

- **Bulk criterion:** the exact criterion the optimality theory (Lecture 8) requires.
- **Threshold:** $\text{threshold} = \theta^2 \mathcal{E}(U, \mathcal{T})^2$; the loop lowers the cutoff γ until elements with $\mathcal{E}_T > \gamma \max_{T'} \mathcal{E}_{T'}$ accumulate that much.
- **Direct parameter:** `adapt.Doerfler_theta` is θ directly.
- **'GR':** recovers uniform refinement: the baseline for comparison.
- **'MS':** (maximum strategy) is cheap (one pass) and effective, but not the bulk criterion the theory requires for optimality.

REFINE

refine_mesh: the outer loop

The outer loop, condensed:

```
function [mesh, uh, fh, n_refined] = refine_mesh(mesh, uh, fh)

n_refined = 0;

first_marked = find(mesh.mark > 0, 1);
while (first_marked)
    n_refined = n_refined + refine_element(first_marked);
    first_marked = find(mesh.mark > 0, 1);
end
```

The driver repeatedly refines the first still-marked element until none remain: one call to `refine_element` per marked element, but each call may recurse.

Recursive bisection with completion

- **Newest-vertex bisection:** REFINE turns the marked set into a new *conforming* mesh by newest-vertex bisection. Bisecting a triangle across its refinement edge (side 3) inserts a vertex at the edge midpoint.

The subtlety

If the neighbour across that edge shares the *same* refinement edge, the two form a *compatible patch* and split together — conforming, done. If not, splitting alone leaves a **hanging node**: the fix is to refine the neighbour *first*, recursively, until a compatible patch is exposed.

refine_element: the recursion

The recursive step, condensed:

```
function n_refined = refine_element(el)
    neighbour = mesh.elem_neighbours(el, 3);           % across the refinement edge
    if (neighbour > 0 && mesh.elem_neighbours(neighbour,3) ~= el) % not compatible
        refine_element(neighbour);                     % refine neighbour first
        neighbour = mesh.elem_neighbours(el, 3);       % may have changed
    end
    % --- now (el, neighbour) is a compatible patch: bisect it ---
    new = new_vertex_at_midpoint(v1, v2);              % v1,v2 = endpoints of side 3
    mesh.elem_vertices(el,:) = [v3 v1 new];            % new vertex becomes vertex 3
    mesh.elem_vertices(child) = [v2 v3 new];           % of both children
    % ... update elem_neighbours / elem_boundaries; interpolate uh, fh; decrement mark ...
end
```

Placing the new vertex as vertex 3 of each child makes its *opposite* edge the new refinement edge automatically — no explicit generation counter needed.

Why does the recursion terminate, and why $n_{\text{refine}} = 2$?

- **Termination:** A consistent initial labeling of $\mathcal{T}_0$ (Lecture 4) guarantees following refinement edges cannot cycle; by the Binev–Dahmen– DeVore/Stevenson bound, the total elements created over the whole run is proportional to the total number of marks — completion is never a bottleneck.
- **Why twice:** Each call bisects a patch once and decrements the mark counter. An element marked *twice* is bisected twice, placing a new node in each edge, decreasing h_T by a factor two.

Validation: adaptive vs. uniform

Closing the loop on the L-shape

- **The benchmark:** Back to the example that motivated adaptivity in Lecture 3:
 $\Omega = (-1, 1)^2 \setminus ([0, 1] \times [-1, 0])$, $u(r, \theta) = r^{2/3} \sin(\frac{2}{3}\theta)$.
- **Uniform stalls:** Uniform refinement stalls at $N_p^{-1/3}$ — degrees of freedom wasted far from the corner.

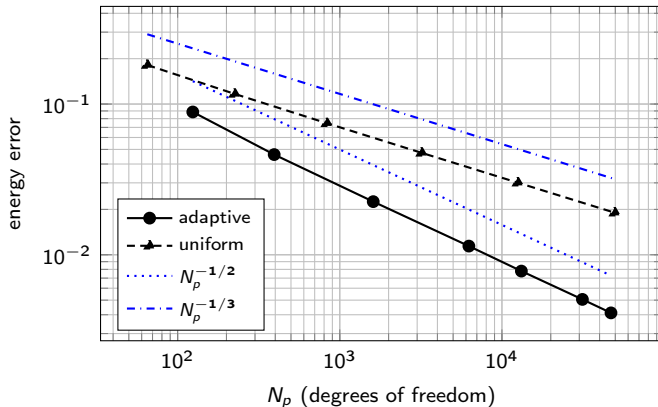
Closing the loop on the L-shape

- **The benchmark:** Back to the example that motivated adaptivity in Lecture 3:
 $\Omega = (-1, 1)^2 \setminus ([0, 1] \times [-1, 0])$, $u(r, \theta) = r^{2/3} \sin(\frac{2}{3}\theta)$.
- **Uniform stalls:** Uniform refinement stalls at $N_p^{-1/3}$ — degrees of freedom wasted far from the corner.

Running the real loop

ESTIMATE with the residual indicator, Dörfler marking at $\theta = 0.5$, recursive bisection: refinement concentrates at the reentrant corner and recovers the **optimal** rate.

Adaptive recovers the optimal rate



Adaptive refinement tracks $N_p^{-1/2}$ — the *smooth-case* rate — even though $u \in H^{1+2/3-\varepsilon}$ only. At $N_p \approx 5 \times 10^4$ the adaptive error is already more than four times smaller than uniform.

Source code

Source code

- **Repository:** Complete Matlab/Octave code and verification scripts:

- <https://github.com/morinpedro/adaptive-fem>



- `afem.m`: driver — the SOLVE–ESTIMATE–MARK–REFINE loop.
 - `init_data.m`: problem data for one of five examples ('L-shape', 'square-Dirichlet', 'big-square-Dirichlet', 'square-mixed', 'kellogg') plus adaptivity parameters.
 - `assemble_and_solve.m`, `estimate.m`, `mark_elements.m`: SOLVE, ESTIMATE, MARK.
 - `refine_mesh.m`, `refine_element.m`: REFINE — recursive newest-vertex bisection with completion.
 - `get_dirichlet_neumann.m`: reconstructs boundary data from `elem_boundaries`.
 - **Mesh-data folders:** four folders, shared across the five examples ('kellogg' reuses `big_square_all_dirichlet/`).

Summary

- ❶ **The loop:** `afem.m` realizes SOLVE–ESTIMATE–MARK–REFINE literally, modules talking only through `mesh`, `uh`, `fh`.
- ❷ **Mesh data:** `elem_neighbours` and `elem_boundaries` give $O(1)$ neighbour lookups — the key enabler for adaptivity.
- ❸ **Estimate:** the residual indicator of Lecture 5, with explicit normal projection of the flux jump to handle discontinuous a .
- ❹ **Mark:** Dörfler/bulk marking realizes the exact bulk criterion the theory needs.
- ❺ **Refine:** recursive bisection with completion; `n_refine=2` delivers the interior node property.
- ❻ **Validation:** on the L-shape, adaptive refinement recovers the optimal $N_p^{-1/2}$ rate that uniform refinement cannot reach.

Summary

- ❶ **The loop:** `afem.m` realizes SOLVE–ESTIMATE–MARK–REFINE literally, modules talking only through `mesh`, `uh`, `fh`.
- ❷ **Mesh data:** `elem_neighbours` and `elem_boundaries` give $O(1)$ neighbour lookups — the key enabler for adaptivity.
- ❸ **Estimate:** the residual indicator of Lecture 5, with explicit normal projection of the flux jump to handle discontinuous a .
- ❹ **Mark:** Dörfler/bulk marking realizes the exact bulk criterion the theory needs.
- ❺ **Refine:** recursive bisection with completion; `n_refine=2` delivers the interior node property.
- ❻ **Validation:** on the L-shape, adaptive refinement recovers the optimal $N_p^{-1/2}$ rate that uniform refinement cannot reach.

Next: Lecture 7 — basic properties and contraction of AFEM.